



5° PROJETO

Disciplina: PAM0466– SISTEMAS INTELIGENTES

Semestre: 2026.1

Docente: PEDRO THIAGO VALÉRIO DE SOUZA

Horário: 3M23 4M45

INTRODUÇÃO E CONTEXTO

A segurança de redes de computadores é um dos desafios centrais da infraestrutura digital moderna. Ataques como negação de serviço (*DoS*), varredura de portas (*Probe*) e acesso remoto não autorizado (*R2L*) causam prejuízos bilionários anualmente e comprometem desde sistemas corporativos até infraestruturas críticas de energia, saúde e comunicações. Sistemas de Detecção de Intrusão (*IDS — Intrusion Detection Systems*) são a principal linha de defesa ativa contra essas ameaças, e a automação dessa detecção por meio de aprendizado de máquina representa a fronteira atual da área.

As características de um ataque *DoS* se misturam às do tráfego legítimo em regiões do espaço de atributos que nenhuma linha reta consegue separar adequadamente. Assim sendo, uma rede Perceptron não é capaz de solucionar esse problema. Assim sendo, o Perceptron Multicamadas (*MLP — Multilayer Perceptron*), com suas camadas ocultas e funções de ativação não-lineares, supera essa limitação, tornando-se uma escolha natural para este tipo de problema.

Neste projeto, você é convidado a construir e treinar uma rede MLP utilizando o PyTorch para classificar conexões de rede como normais ou ataques, a partir de atributos extraídos de pacotes de rede.

CONJUNTO DE DADOS

O projeto utiliza o *dataset* NSL-KDD (<https://www.kaggle.com/datasets/hassan06/nslkdd>), disponibilizado pelo Canadian Institute for Cybersecurity da Universidade de New Brunswick. O NSL-KDD é uma versão refinada do histórico KDD Cup 1999.

O conjunto disponibiliza dois arquivos: *KDDTrain+.txt* (125.973 amostras) e *KDDTest+.txt* (22.544 amostras). Cada registro representa uma conexão de rede e possui 41 atributos divididos em três grupos: (i) básicos — duração, protocolo (*protocol_type*), serviço (*service*), *flag* de estado da conexão (*flag*), bytes transferidos etc.; (ii) de conteúdo — número de tentativas de *login* com falha, acesso a arquivos sensíveis, comandos executados em sessões shell etc.; (iii) de tráfego — estatísticas das últimas conexões ao mesmo host e ao mesmo serviço, como taxa de erros SYN e taxa de conexões rejeitadas. Dos 41 atributos, 38 são numéricos e 3 são categóricos (*protocol_type*, *service* e *flag*).

O rótulo original possui 23 tipos de ataque agrupados em 4 categorias (*DoS*, *Probe*, *R2L*, *U2R*) além da classe *normal*; neste projeto, o problema será tratado como classificação binária: 0 para conexão normal e 1 para qualquer tipo de ataque.

ATIVIDADES

Realize as seguintes tarefas utilizando o *dataset* fornecido:

- 1 Preparação dos dados: carregue os arquivos *KDDTrain+.txt* e *KDDTest+.txt* com o *pandas*, atribuindo nomes às colunas a seguinte lista de rótulos:

```
col_names = [  
    "duration", "protocol_type", "service", "flag", "src_bytes",  
    "dst_bytes", "land", "wrong_fragment", "urgent", "hot",
```

```

        "num_failed_logins", "logged_in", "num_compromised", "root_shell",
        "su_attempted", "num_root", "num_file_creations", "num_shells",
        "num_access_files", "num_outbound_cmds", "is_host_login",
        "is_guest_login", "count", "srv_count", "serror_rate",
        "srv_serror_rate", "rerror_rate", "srv_rerror_rate", "same_srv_rate",
        "diff_srv_rate", "srv_diff_host_rate", "dst_host_count",
        "dst_host_srv_count", "dst_host_same_srv_rate",
        "dst_host_diff_srv_rate", "dst_host_same_src_port_rate",
        "dst_host_srv_diff_host_rate", "dst_host_serror_rate",
        "dst_host_srv_serror_rate", "dst_host_rerror_rate",
        "dst_host_srv_rerror_rate", "attack_type", "difficulty_level"
    ]

```

As duas últimas colunas refere-se à saída (tipo de ataque e nível de dificuldade, mas essa última deve ser descartada). Aplique *one-hot encoding* nos três atributos categóricos (`protocol_type`, `service`, `flag`). Em seguida, binarize o rótulo (normal $\rightarrow$ 0; qualquer ataque $\rightarrow$ 1) e normalize todos os atributos numéricos com Z-score usando apenas as estatísticas do conjunto de treino.

- 2 Implementação da MLP em PyTorch: crie uma classe MLP herdando de `nn.Module`. A função de perda deve ser `BCEWithLogitsLoss`. Implemente o método `forward` e instancie o modelo, o otimizador Adam e a função de perda. Encapsule os dados em `TensorDataset` e utilize um `DataLoader` com *mini-batches* de tamanho 256.
- 3 Treinamento e monitoramento: treine a MLP por 50 épocas registrando a *loss* de treino a cada época. Reserve 20% do conjunto de treino como validação (sem embaralhamento após a divisão) e registre também a *loss* de validação. Ao final, plote as duas curvas na mesma figura. A escolha topológica da rede (número de neurônios por camada oculta, número de camadas ocultas e função de ativação) é de sua responsabilidade. Para isso, recomenda-se testar várias topologias de rede, verificando-se o desempenho no conjunto de validação. Verifique a necessidade de parada antecipada durante o processo de treinamento. **Atenção:** para o neurônio da camada de saída, não utilize função de ativação, pois será usada a função de perda `BCEWithLogitsLoss`.
- 4 Avaliação final: utilizando a arquitetura de melhor desempenho na validação, avalie o modelo no conjunto de teste `KDDTest+.txt`. Aplique um limiar de 0,5 sobre a saída da sigmoide para obter os rótulos preditos e gere a matriz de confusão. Calcule e reporte acurácia, precisão, *recall* e F1-score. Em um sistema de detecção de intrusão em produção, qual tipo de erro é mais crítico: um falso negativo (ataque classificado como normal) ou um falso positivo (conexão normal classificada como ataque)? Como essa escolha afeta a definição do limiar de decisão?

Pede-se, como entrega, os códigos utilizados para análise e construção do modelo, disponibilizados em um repositório público (GitHub, GitLab, etc.), juntamente com um relatório contendo as interpretações dos resultados e as conclusões sobre a adequação da rede Adaline para este problema.